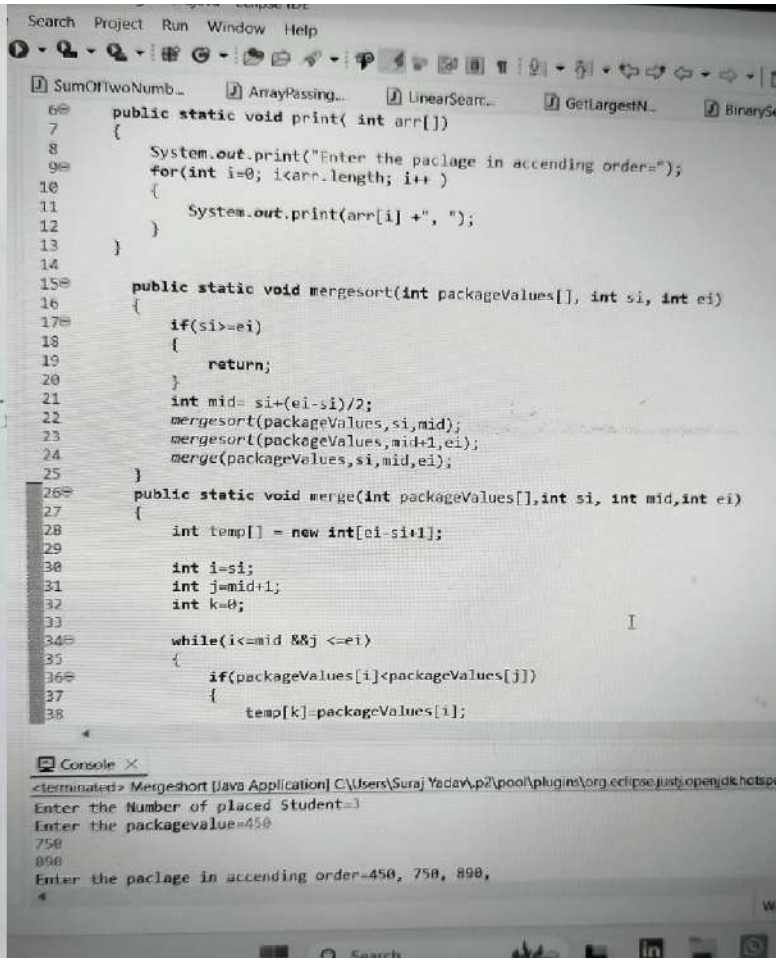
```
import java.util.*;
Public class MergeShort
{

    Public static void print(int arr[])
    {
      System.out for(int i=0; i<arr.length; i++)
                 {
                 System.out.print(arr[i]+" , ");
                 }
    }

  Public static void mergsort(intarr[], intsi, int ei)
  {
     if(si>=ei)
     {
       return;
     }
     int mid = si+(ei+si)/2;
     mergeshort(arr, si, mid);
     mergeshort(arr, mid+1, ei);
     merge(arr, mid, si, ei);
  }
Public static void merge(int packagevalue[], intsi
                         int mid, int ei)
 {

     int temp[] = new int(ei-si+1);

      int i = si;
      int j = mid+1;
```

```
  while(i<=mid && j<=ei)
{
   if(packageValues[i] < packageValue[j])
   {
       temp[k] = packageValues[i];
         k++;
         i++;
   }
   else
    {
      tem[k] = packagevalues[j];
    }
}
while(i<=mid)
  {
     temp[k] = packageValues[i];
        i++;
        k++;
  }
while(j<=ei)
  {
     temp[k] = packa values[j];
        j++;
     }   k++;
 for(k=0; j=si; k<tem.length; k++, i--)
  {
     PackageValues[i) = temp[k];
  }
}
```

```
Public static void main(String[] args)
{
    Scanner Sc = new Scanner(System.in);
    System.out.print("Enter the number of placed
                      Student=");
    int N = Sc.nextInt();
    System.out.print("Enter the Package Values=");
    int packageValues = new int[N];
for(int i=0; i<N; i++)
{
    PackcaValues[i] = Sc.nextInt();
}
mergeSort(Package values, 0, N-1);
 print(packageValues);
}
}
```



Question No 4(b)

```
import java.util.*;
Public Class ProcessingTime
{
  Public Static void merge(int times[], int si, int mid,
                           int ei)
  {
    int temp[] = new int[ei-si+1];
    int i=si;
    int j = mid+1;
    int k = 0;
  while(i<=mid && j<=ei)
   {
     if (times[i]<=times[j])
     {
       tem[k] = times[i];
         i++;
         k++;
     }
     else
      { tem[k] =times[j];
          k++;
          j++;
      }
   }
  while(i<=mid)
   {
      temp[k] = times[i];
        k++;
        i++;
```

```
   {
      temp[k++] = times[j++];
   }
  for(k=0; k<temp.length; k++)
   {
      times[si+k] = temp[k];
   }
}
Public Static void mergesort(int times[], int si,int ei
 {
    if (si>=ei)
    {
      return
    }
  int mid = si+(ei-si)/2;
  mergesort(~~si, arr~~ times, si, mid);
  mergesort (times, mid+1, ei);
  merge (times, si, mid, ei);
}
Public Static void main(String[] args)
{
  Scanner sc = new Scanner(System.in);
  System.out.print("Enter the Integer N=");
    int N= sc.nextInt();
    int[] times = new int[N];
  System.out.print("Enter the N space-separated
                    processing times=");
```

```
        {
            times[i] = sc.nextInt();
        }
        mergesort(times, 0, N-1);
    System.out.println("Sorted processing Time:");
        for(int val : times)
        {
            System.out.print(val+" ");
        }
        System.out.println();

        double median;
        if(N%2 == 1)
        {
            median = times[N/2];
        }
        else
        {
            median = (times[N/2-1]+times[N/2])/2.0;
        }
    System.out.println("median Processing Time:"+median);

        int count=0;
        for(int val: times)
        {
            if(val>median)
            count++;
```



```
    System.out.println("order greater than median:"+count);

        int diff = times[N-1] - times[0];
        System.out.println("Difference(max-min)"
                +diff);
    }
}
```

```
SumOfTwoNumb...  ArrayPassing...  LinearSearc...  GetLargestN...  BinarySearc...
 1 package Divideandconcer;
 2     import java.util.*;
 3
 4     public class ProcessingTime {
 5
 6
 7         public static void merge(int[] times, int si, int mid, int ei
 8             int[] temp = new int[ei - si + 1];
 9             int i = si, j = mid + 1, k = 0;
10
11             while (i <= mid && j <= ei) {
12                 if (times[i] <= times[j]) {
13                     temp[k++] = times[i++];
14                 } else {
15                     temp[k++] = times[j++];
16                 }
17             }
18             while (i <= mid) temp[k++] = times[i++];
19             while (j <= ei) temp[k++] = times[j++];
20
21             for (k = 0; k < temp.length; k++) {
22                 times[si + k] = temp[k];
23             }
```

Console X

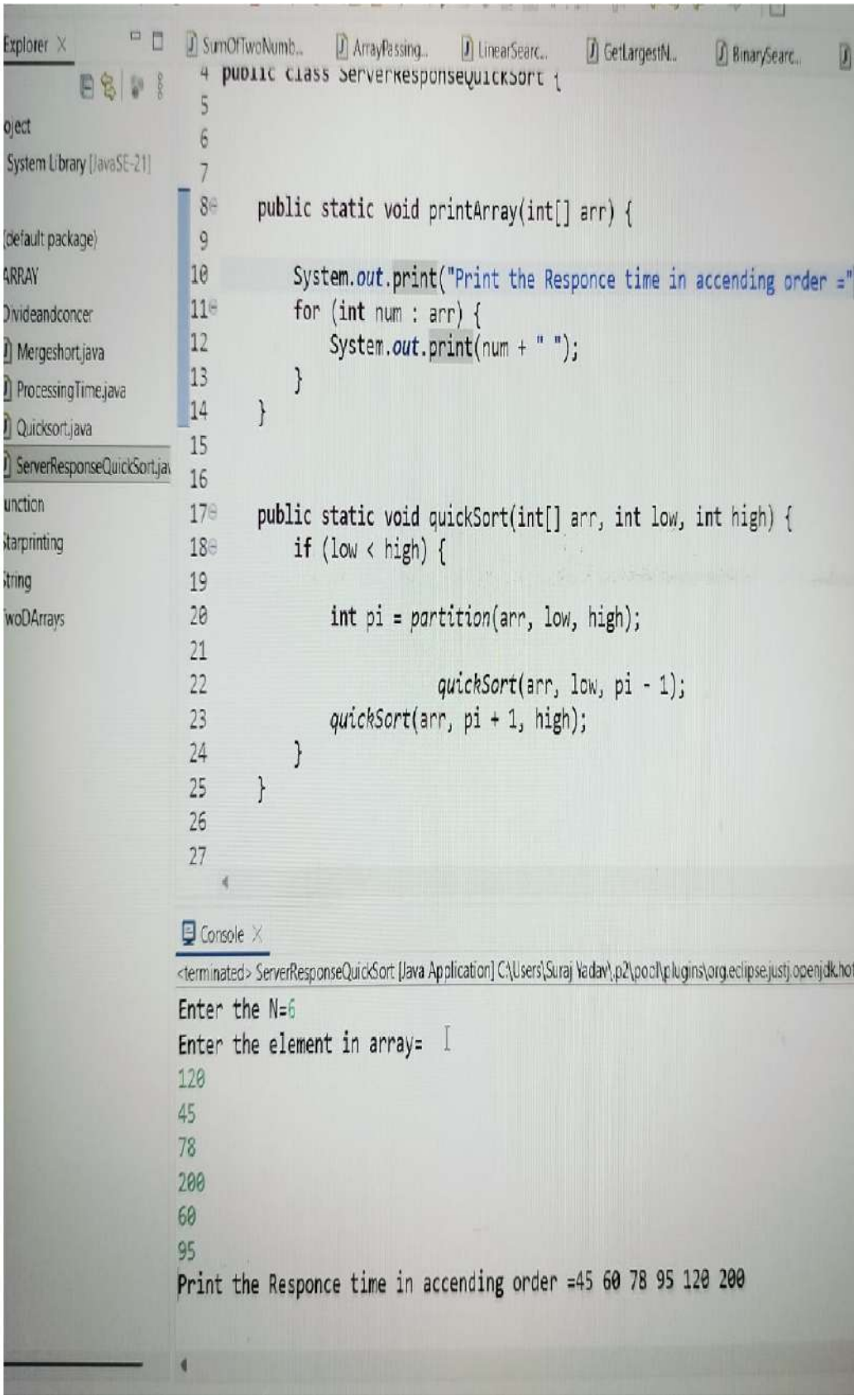
<terminated> ProcessingTime [Java Application] C:\Users\Suraj Yadav\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot

```
Enter the Integer N=5
Enter the  N space-separated processing times.=23
45
32
56
78
Sorted Processing Times:
23 32 45 56 78
Median Processing Time: 45.0
Orders greater than median: 2
Difference (Max - Min): 55
```

QuickSort 2 Question-1

```
import java.util.*;
Public class ServerResponseQuickSort
{
   Public static void printARRAY(int arr[])
   {
    System.out.print("Print the Responce time in
                      according order=");
    for(int num : arr)
      {
        System.out.print(num+" ");
      }
   }
. Public static void quicksort(int arr[], int low, int
                                high)
  {
    if(low>=high)
      { return;
      }
    int pi = partition(arr, low, high);
     quicksort(arr, low, pi-1);
     quicksort(arr, pi+1, high);
  }
. Public Static int partition(int arr[], int low
                              int high);
  {
     int pivot = high;
     int i = low-1;
```

```
for(int j=low; j<high; j++)
 {
  if(arr[j]<=pivot)
   {
    i++;
    int temp = arr[i];
      arr[i] = arr[j];
       arr[j] = temp;
   }
 int temp = arr[i+1];
  arr[i+1] = arr[high]
   arr[high] = temp;
  return i+1;
 }
Public Static void main(String []args)
 {
  Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int arr[] = new int[n];
    for(int i=0; i<n; i++)
      {
        arr[i] = sc.nextInt();
      }
    quicksort(arr,0, n-2);
     printARRAY(arr);
  }
}
```



Quicksort Question 2.(B)

```
import java.util.*;
Public Class TradeValuesQuickSort
{
  Public Static void quick(int[], arr, int low,
                           int high)
  {
    if(low<high)
    {
      int pi = partition(arr,low,high);
       quicksort(arr,low, pi-1);
       quicksort(arr, pi+1, high);
    }
  }
Public static int partition(int arr[], int low
                            int high)
{
   int pivot = arr[high];
     int i = low-1;
  for(int j=low; j<high; j++)
   {
     if (arr[j] >= pivot)
     {
       i++;
      int temp = arr[i];
        arr[i] = arr[j];
         arr[j] = temp;
     }
  }
```

```
  int temp = arr[i+1];
   arr[i+1] = arr[high];
    arr[high] = temp;
   return i+1;
}
Public Static void main(String args[])
{
  Scanner sc = new Scanner(System.in)
   int N = sc.nextInt();
   int trades[] = new int[N];
  for(int i=0; i<N; i++)
   {
     trade[i] = sc.nextInt();
   }
  quick sort(trades, 0, N-1);
System.out.println("Sorted trades values(Decending)")
for(int val: trades)
 {
   System.out.print(val+" ");
 }.
 System.out.println();
System.out.println(" Top 5 Trade values");
   int limit = Math.min(5,N);
    int SumTop5=0;
```

Search Project Run Window Help

SumOfTwoNumb... ArrayPassing... LinearSearc... GetLargestN...

```
1  package Divideandconcer;
2
3  import java.util.*;
4
5  public class TradeValuesQuickSort {
6
7      // Quick Sort function
8      public static void quickSort(int[] arr, int low, int high) {
9          if (low < high) {
10             int pi = partition(arr, low, high);
11             quickSort(arr, low, pi - 1);
12             quickSort(arr, pi + 1, high);
13         }
14     }
15
16     // Partition method (descending order)
17     public static int partition(int[] arr, int low, int high) {
18         int pivot = arr[high];
19         int i = low - 1;
20
21         for (int j = low; j < high; j++) {
```

Console

<terminated> TradeValuesQuickSort [Java Application] C:\Users\Suraj Yadav\.p2\pool\plugins\org.ecl

```
Enter the Integer N=8
Enter the Element in array=500
1200
700
2000
900
1500
3000
2500
Sorted Trade Values (Descending):
3000 2500 2000 1500 1200 900 700 500
Top 5 Trade Values:
3000 2500 2000 1500 1200
Average of Top 5: 2040.0
Overall Average: 1537.5
Count > Overall Average: 3
```